Event Intelligence — دليل فهم المشروع للمناقشة

1. الفكرة العامة (Big Picture)

المشروع عبارة عن **Real-time data pipeline** بسيط:

```
[صفحة الطلب HTML]
      ↓ POST /order (JSON)
[FastAPI - producer.py]
      ↓ ينشر رسالة على Kafka
[Kafka topic: test-topic]
      ↓
      ├──→ [Flink job] يقرأ الرسالة، يحوّلها UPPERCASE، وينشرها على [output-topic]
      |          ↓
      |    [consumer.py] يقرأ من output-topic ويخزّن بـ Postgres (جدول
kafka_messages)
      |
      └──→ (نفس test-topic) [agent.py] يقرأ الرسالة الأصلية، يبعتها لـ Ollama (LLM محلي)
                يحصل على تحليل، ويخزّنه بـ Postgres (جدول agent_results)

[Dashboard HTML] ← GET requests → [FastAPI] ← SELECT → [Postgres]
```

النقطة المهمة اللي لازم تفهمها وتحكيها بالمناقشة: فيه **مسارين متوازيين (two parallel consumers)** يقرأو من نفس topic (`test-topic`) بشكل مستقل — واحد بيروح عبر Flink للتحويل والتخزين العادي، والتاني (agent) بيروح مباشرة لتحليل بالـ AI. هيك هو نموذج مصغّر عن Event-Driven Architecture حيث نفس الـ event ممكن يشغّل أكتر من consumer مستقل عن بعضهم.

---

2. FastAPI — `scripts/producer.py`

هاد الـ Entry Point الأساسي، مسؤول عن استقبال الطلبات من صفحة الويب ونشرها على Kafka.

الـ Models (Pydantic)

```python
class OrderItem(BaseModel):
    product_id: str
    product_name: str
    quantity: int
    unit_price: float

class OrderRequest(BaseModel):
    branch: str
    customer_name: str
    payment_method: str
    notes: Optional[str] = ""
    items: List[OrderItem]
```

هاي بتعرّف شكل الـ JSON المتوقع بجسم الطلب. FastAPI بيتحقق تلقائيًا من الأنواع (validation) — لو حقل ناقص أو نوعه غلط، بيرجع error 422 تلقائيًا بدون ما نكتب أي كود تحقق يدوي.

الاتصال بـ Kafka

```python
```

```python
producer = None

def get_producer():
    global producer
    if producer is None:
        producer = KafkaProducer(
            bootstrap_servers=["kafka:29092"],
            value_serializer=lambda v: json.dumps(v).encode("utf-8")
        )
    return producer
```

هاد نمط **Lazy Singleton** — الـ KafkaProducer ما بينعمل إلا أول مرة تنستدعى `get_producer()`، وبعدين بينحفظ بمتغيّر global وبينعاد استخدامه لكل الطلبات الجاية (تفادي فتح اتصال جديد كل مرة، لأنه فتح اتصال Kafka عملية مكلفة).

`value_serializer` بيحوّل الـ dict لـ JSON string وبعدين لـ bytes، لأنه Kafka بالأساس بينقل bytes بس.

تحويل الطلب لرسالة نصية

```python
def build_order_message(order: OrderRequest) -> str:
    items_summary = ", ".join([f"{item.product_name} x{item.quantity}" for item i
    return (
        f"Order from {order.customer_name} | "
        f"Branch: {order.branch} | "
        f"Payment: {order.payment_method} | "
        f"Items: {items_summary} | "
        f"Notes: {notes_part}"
    )
```

هاي أهم نقطة تصميمية بالمشروع كله. بدل ما يبعت JSON منظّم لـ Kafka، المشروع بيحوّل الطلب لسطر نص واحد بصيغة ثابتة (pipe-delimited: `|`). هاد قرار بسيط ومقصود عشان:

1. Flink يقدر يعمل عليه بسهولة (تحويل نص كامل) `UPPER()`.
2. `dashboard_utils.py` لاحقًا بيقدر يعمل **parse** له بـ regex ويرجعه لبيانات منظمة.

هاد تصميم بسيط جدًا (ومو أفضل ممارسة بالإنتاج الحقيقي — الأفضل نبعت JSON structured من الأساس ونتفادى Regex Parsing لاحقًا)، بس منطقي لغرض تعليمي/PoC.

الـ Endpoint الرئيسي

```python
@app.post("/order")
def create_order(order: OrderRequest):
    message_text = build_order_message(order)
    message = {"data": {"message": message_text}, "timestamp": time.time()}
    kafka_producer = get_producer()
    future = kafka_producer.send(TOPIC_NAME, value=message)
    result = future.get(timeout=10)
    return {"status": "success", "partition": result.partition, "offset": result.
```

- `kafka_producer.send()` عملية **async** بطبيعتها بترجع `future` فورًا.
- `future.get(timeout=10)` بيحوّلها لعملية **sync** (استلم الرسالة فعليًا Kafka بينتظر حتى 10 ثواني للتأكد إنه)، "لأنه لازم نرجع للمستخدم تأكيد حقيقي إنه الطلب انبعت، مش بس أرسلناه ونتمنى يوصل" وهاد مهم للـ API.
- `result.partition` و `result.offset` هني إثبات فعلي إنه Kafka خزّن الرسالة (كل رسالة إلها موقع محدد جوا الـ partition).

الـ endpoints التانية (لخدمة الداشبورد)

```python
```

```
@app.get("/get_metrics")
@app.get("/latest_orders")
@app.get("/sales_branch")
@app.get("/branch_performance")
```

كل هدول بينادو `fetch_dashboard_data()` من `dashboard_utils.py` (يلي بيسحب من Postgres)، وبعدين يمررو النتيجة لدالة معالجة مختلفة. لاحظ إنه **كل استدعاء بيعمل query جديد كامل لكل الجدول** — مو محسّن، بس بسيط ومباشر لعدد بيانات صغير متل هاد المشروع.

---

3. Kafka — دوره بالمشروع

Kafka هون هو الـ **message broker** اللي بيفصل بين مين بينشر البيانات (Producer) ومين بيستهلكها (Consumers)، وبخليهم مايعرفوش ببعض إطلاقًا.

الفكرة الأساسية اللي لازم تحكيها:

- **Topic** = قناة/مجرى بيانات بموضوع محدد (عنا: `test-topic` و `output-topic`).
- **Producer** بس بيعرف يكتب لتوبيك، **Consumer** بس بيعرف يقرأ منه — ما في ارتباط مباشر بينهم.
- Kafka بيحتفظ بالرسائل حتى لو محدا قاعد يقرأها هلق (durable log) — لهيك لما نشغّل consumer جديد بـ `auto_offset_reset='earliest'`، بيقدر يرجع يقرأ كل الرسائل القديمة من البداية.
- **Consumer Group:** لو صار عندك أكتر من consumer بيسجل نفسه تحت `group_id`. بيوزع الـ Kafka partitions عليهم (Load Balancing). بمشروعنا كل consumer إله group_id مختلف (`consumer-output-topic`, `flink-consumer-group`, `agent`) — يعني كل واحد فيهم **بياخد نسخة كاملة مستقلة من كل الرسائل**، مش تقسيم فيما بينهم. هاد الفرق المهم بين "Consumer Groups مختلفة" و"نفس الـ Group".

Zookeeper

`cp-zookeeper` — مسؤول عن إدارة الـ Kafka بإصدارات الأقدم (متل هاد المشروع، 7.6.0)، Zookeeper بيخزن metadata (مين هو الـ leader لكل partition، إدارة الـ cluster state). الإصدارات الأحدث من Kafka (KRaft mode) ما عاد في حاجة لـ Zookeeper، بس هاد المشروع لسا عم يستخدم الطريقة التقليدية.

`kafka-init` service

```yaml
command: sleep 60 && kafka-topics --create --if-not-exists --topic test-topic ...
```

كونتينر بيشتغل مرة وحدة بس (`"restart: "no`)، مهمته ينشئ الـ (`test-topic`, `output-topic`) topics بمجرد ما Kafka يجهز، وبعدها يطلع (Exit). هاد بديل عن "auto topic creation" (يلي ممكن يسبب مشاكل لو صار أول اتصال بالتوبيك قبل ما يكون معرّف صح).

Listeners (الاثنين (نقطة كتير مهمة بالمقابلات/المناقشات

```yaml
KAFKA_ADVERTISED_LISTENERS: PLAINTEXT://kafka:29092,PLAINTEXT_HOST://localhost:90
```

Kafka هون مفتوح بمنفذين لسببين مختلفين:

- `kafka:29092` → **جوا نفس شبكة** Docker يستخدمه أي كونتينر (producer-api, consumer, flink-job, agent) — بيوصلو لـ Kafka باسمه كـ hostname داخل الشبكة.
- `localhost:9092` → لو حابب توصل لـ Kafka **من جهازك مباشرة** (مثلًا أداة GUI أو سكربت تشغّله برا Docker).

---

4. Consumer — `scripts/consumer.py`

أبسط جزء بالمشروع منطقيًا. Loop لا-نهائي:

```python
```

```
consumer = KafkaConsumer(
    TOPIC_NAME,                          # output-topic (بعد ما Flink يحوّلها)
    bootstrap_servers=['kafka:29092'],
    auto_offset_reset='earliest',
    enable_auto_commit=True,
    group_id=f'consumer-{TOPIC_NAME}',
    value_deserializer=lambda x: json.loads(x.decode('utf-8'))
)

for message in consumer:             # blocking loop — بينتظر كل رسالة جديدة
    insert_message(conn, message.value)
```

- `for message in consumer` — هاد الـ pattern القياسي بمكتبة kafka-python: بيفتح اتصال ويضل ينتظر (block) لحد ما توصل رسالة، وبعدين يكمل الـ loop تلقائيًا.
- `enable_auto_commit=True` يعني Kafka بيسجل "لغاية وين قرأت" أوتوماتيكيًا (offset) بعد فترة، فلو الكونتينر بيخليه ياخد كل التاريخ `earliest` (إلا الأولى مرة، وقتها) وقع وبلش من جديد، ما رح يعيد قراءة نفس الرسائل يلي قراها قبل.
- كل رسالة توصل بتنكتب مباشرة بجدول `kafka_messages` (نفس الجدول يلي منه الداشبورد بيقرا).

ملاحظة: هاد الملف مصمم يقرا `output-topic` (بعد التحويل من Flink)، مش `test-topic` الخام. يعني لو Flink مش شغال، هاد الـ consumer ما رح يستقبل أي إشي أبدًا حتى لو Kafka نفسه تمام.

---

5. Apache Flink — `flink-job/flink_job.py`

هون أعقد جزء بالمشروع، وغالبًا بيسألوك عنه بالمناقشة، فخليني افصّل.

شو هو Flink أصلًا؟

محرك معالجة بيانات **Stream Processing** — عكس Batch processing (يلي بيعالج كتلة بيانات كاملة مرة وحدة وبيخلص)، Flink مصمم يعالج البيانات **بشكل مستمر لحظة ما توصل**، بدون توقف.

Table API بدل DataStream API

المشروع مستخدم **Flink SQL / Table API** (مش الـ DataStream API الأقل تجريدًا). يعني بدل ما نكتب كود Python يعالج كل event يدوي، منعرّف **جداول** (تشبه جداول SQL عادية) ومنكتب استعلام SQL عادي، و Flink بيدير التنفيذ المستمر تلقائيًا بالخلفية.

تعريف جدول المصدر (Source Table)

```sql
CREATE TABLE kafka_source (
    data ROW<message STRING>,
    `timestamp` DOUBLE
) WITH (
    'connector' = 'kafka',
    'topic' = 'test-topic',
    'properties.bootstrap.servers' = 'kafka:29092',
    'scan.startup.mode' = 'earliest-offset',
    'format' = 'json',
    'json.ignore-parse-errors' = 'true'
)
```

هاد بالضبط بيوصف شكل رسالة الـ JSON يلي جاية من الـ producer:

```json
{"data": {"message": "Order from Ahmad | ..."}, "timestamp": 1234567.89}
```

لاحظ `<data ROW<message STRING` — هاد بيمثّل الـ nested object، و Flink SQL بيقدر يوصله بـ dot notation: `data.message`.

تعريف جدول الوجهة (Sink Table)

```sql
```

```sql
CREATE TABLE kafka_sink (
    original_message STRING,
    transformed_message STRING,
    processed_at TIMESTAMP(3)
) WITH ('connector' = 'kafka', 'topic' = 'output-topic', 'format' = 'json')
```

الاستعلام (Continuous Query)

```sql
INSERT INTO kafka_sink
SELECT
    data.`message` AS original_message,
    UPPER(data.`message`) AS transformed_message,
    CURRENT_TIMESTAMP AS processed_at
FROM kafka_source
WHERE data.`message` IS NOT NULL
```

هاد مو استعلام بيتنفذ مرة وحدة ويخلص متل SQL عادي — هاد Flink بيضل **Continuous/Unbounded Query**: مشغّل هاد المنطق على كل رسالة جديدة توصل لـ `kafka_source` **إلى ما لا نهاية**، لحد ما توقف الـ job يدويًا. هون التحويل الوحيد هو `UPPER()`، بس بالمبدأ ممكن تحط أي منطق SQL (windowing، aggregation، JOIN... إلخ).

نقطة مهمة اكتشفناها سوا (لازم تذكرها بالمناقشة!)

```python
env = StreamExecutionEnvironment.get_execution_environment()
```

لو شغّلت هاد بـ `python3 flink_job.py` مباشرة، PyFlink **ما بيتصل بالـ Flink Cluster الحقيقي** (jobmanager+taskmanager) — عوضًا عن هيك بيسوي **mini-cluster محلي مدمج جوا نفس البروسيس**، فالـ job بيشتغل ويعالج البيانات فعليًا، بس **بشكل معزول تمامًا** وما بيظهر إطلاقًا على Flink Dashboard (8081).

الحل الصحيح: تشغيل الـ job عن طريق أداة الـ CLI الرسمية لـ Flink:

```dockerfile
CMD ["/opt/flink/bin/flink", "run", "-d", "-py", "flink_job.py", "-m", "jobmanage
```

- `flink run` = أداة تقديم (submit) الـ job لـ session cluster موجود، بدل تشغيله محليًا.
- `-m jobmanager:8081` = عنوان الـ REST API حق jobmanager (اللي بيدير توزيع الشغل على taskmanager).
- `-d` = detached mode: قدّم الـ job وارجع فورًا (الكونتينر بيطلع Exited(0) — الـ job بنجاح لأنه دوره خلص نفسه بيضل شغال جوا jobmanager/taskmanager).

jobmanager vs taskmanager (فرق كلاسيكي لازم تعرفه)

- **jobmanager:** "المايسترو" — بيستقبل الـ jobs، بيوزعها، بيتابع حالتها، بيوفر REST API + الداشبورد (8081).
- **taskmanager:** "العامل الفعلي" — "عنده Task Slots (slot=2، هون معرفين) وهو يلي فعليًا بيشغّل كود المعالجة ويقرا/يكتب من Kafka.

---

6. AI Agent — `scripts/agent.py`

نفس فكرة `consumer.py` لكن بيقرا من `test-topic` مباشرة (مش من output-topic)، وبدل ما يخزّن مباشرة، بيمرر الرسالة لموديل AI محلي أولاً:

```python
def analyze_with_ollama(message):
    prompt = f"...Analyze this order...\nOrder: {message}\n..."
    response = requests.post(OLLAMA_URL, json={'model': OLLAMA_MODEL, 'prompt': p
    return response.json().get('response', 'No analysis available')
```

- **Ollama** هو runtime لتشغيل LLM models محليًا على جهازك (متل llama3) — بديل عن استخدام OpenAI API مثلًا، فكل شي يضل محلي بدون إنترنت أو تكلفة API.

- `config/agent_config.yaml` — الإعدادات (اسم الموديل، الرابط) موجودة بالكود، مفصولة بملف hardcoded (Separation of config from code).
- **نقطة كتير مهمة:** الـ agent شغال **جوا كونتينر Docker**، لكن Ollama شغال **على جهازك مباشرة** (host machine)، بيشاور على الكونتينر نفسه مش جهازك! لهيك لازم `localhost` مش جوا كونتينر. من جوا الكونتينر، نستخدم `host.docker.internal` (اسم خاص بيوفره Docker المضيف بيشاور دايمًا عالجهاز المضيف)، وهاد يلي عرّفناه بـ:

```yaml
extra_hosts:
  - "host.docker.internal:host-gateway"
```

النتيجة بتنخزن بجدول منفصل `agent_results` (مش نفس جدول الـ consumer العادي) — لأنه هاد "تحليل إضافي" مستقل عن مسار المعالجة الأساسي.

---

7. PostgreSQL — قاعدة البيانات

`sql/init.sql`

```sql
```

```sql
CREATE TABLE IF NOT EXISTS kafka_messages (
    id SERIAL PRIMARY KEY,
    original_message TEXT,
    transformed_message TEXT,
    processed_at TIMESTAMP,
    inserted_at TIMESTAMP DEFAULT CURRENT_TIMESTAMP
);

CREATE TABLE IF NOT EXISTS agent_results (
    id SERIAL PRIMARY KEY,
    original_message TEXT,
    agent_analysis TEXT,
    analyzed_at TIMESTAMP DEFAULT CURRENT_TIMESTAMP
);
```

هاد الملف بينحط بمسار خاص جوا كونتينر Postgres الرسمي:

```yaml
volumes:
  - ./sql/init.sql:/docker-entrypoint-initdb.d/init.sql
```

الصورة الرسمية لـ Postgres عندها آلية: أي ملف `.sql` أو `.sh` موجود بمجلد `/docker-entrypoint-initdb.d/` بينفَّذ **تلقائيًا وبس مرة وحدة**، أول ما الـ container يشتغل لأول مرة على data volume فاضي. لهيك أي مرة تعمل `docker compose down -v` (تمسح الـ volumes)، الجداول بتتصلق من جديد تلقائيًا.

الفرق بين البورت 5432 و 5433

```yaml
ports: ["5433:5432"]
```

جوا شبكة Docker، Postgres شغال طبيعي على بورته الافتراضي 5432 (هيك بيتوصلو له باقي الكونتينرات). بس على جهازك (localhost)، حجزنا بورت مختلف 5433 — على الأغلب لتفادي تعارض مع أي Postgres تاني ممكن يكون مثبّت أو شغال عندك مباشرة على 5432.

---

8. `dashboard_utils.py` — تحويل النص الخام لبيانات منظمة

هاد أهم ملف "منطق أعمال" (business logic) بالمشروع. بيسحب الصفوف الخام من `kafka_messages` وبيحوّلها لـ JSON منظم للداشبورد.

إعادة البناء بـ Regex

```python
pattern = r"Order from (.*?) \| Branch: (.*?) \| Payment: (.*?) \| Items: (.*?) \
match = re.match(pattern, original_message)
```

لأنه الرسالة الأصلية مخزّنة كنص عادي (مش JSON columns)، هون بنرجع "نفك" هاد النص لحقول منفصلة (customer, branch, payment, items, notes) — تمامًا عكس العملية يلي صارت بـ `build_order_message()` بالـ producer. هاد بالضبط سبب قولنا قبل شوي إنه تخزين نص خام "flat string" قرار بسيط بس مكلف لاحقًا — لأنه أي تغيير بصيغة الرسالة بالـ producer لازم ينعكس فورًا هون بالـ regex، وإلا الـ parsing بيفشل بصمت (silent failure، بيرجع "Unknown" بس).

حساب الأسعار

```python
PRODUCT_PRICES = {"Anker Power Bank": 120, "Lenovo IdeaPad Laptop": 3200, ...}
```

ملاحظة مهمة: الأسعار **مو مخزّنة أبدًا** بالرسالة أو بقاعدة البيانات! هي **hardcoded بالكود** هون فقط، وبتنحسب لاحقًا وقت العرض بس (`quantity * unit_price`). يعني السعر الحقيقي وقت الطلب مش موثّق فعليًا بأي مكان — لو غيّرت سعر منتج بالمستقبل، الطلبات القديمة رح يتغيّر سعرها المعروض بأثر رجعي كمان (bug تصميمي محتمل يستاهل تذكره كنقطة نقد بالمناقشة).

دوال التجميع (Aggregation)

- `calculate_metrics()`: مجموع الطلبات، الإيرادات، عدد المنتجات، عدد الفروع، وآخر تحديث.
- `get_latest_orders()`: أحدث 10 طلبات (sorted by timestamp).
- `get_sales_by_branch()`: مجموع مبيعات كل فرع.
- `get_branch_performance()`: إحصائيات تفصيلية لكل فرع + متوسط قيمة الطلب.

كل هاي الدوال بتشتغل بـ **Python بالذاكرة** بعد ما نجيب كل الصفوف (مش SQL aggregation بقاعدة البيانات نفسها زي `GROUP BY` أو `SUM()`) — منطقي لعدد صفوف صغير، بس مش efficient لو صار عندك آلاف الصفوف (نقطة تحسين محتملة تذكرها لو سألوك "كيف تحسّن الأداء؟").

---

9. الواجهات (Frontend)

`customer-order-page/` — صفحة إنشاء طلب

`script.js` بيبني JSON مطابق تمامًا لشكل `OrderRequest` بالـ FastAPI، وبيبعته بـ:

```
js

fetch("http://localhost:5000/order", { method: "POST", ... })
```

`dashboard/` — لوحة المتابعة

`script.js` بيسحب من 4 endpoints (`/get_metrics`, `/sales_branch`, `/branch_performance`, `/latest_orders`) بشكل مستقل، كل وحدة بتعبي جزء مختلف من الواجهة (كروت الإحصائيات، جداول، رسوم بيانية).

كلا الملفين معرفين `API_BASE_URL = "http://localhost:5000"` بشكل ثابت — يعني الصفحتين لازم تنفتحو من المتصفح مباشرة (مش من جوا كونتينر)، وبيتواصلو مع الـ FastAPI عبر البورت المفتوح على جهازك.

---

10. الـ Docker Compose — التنسيق (Orchestration)

`Dockerfile` الرئيسي (لخدمات: producer-api, consumer, db-seed, agent)

```
dockerfile
```

```
FROM python:3.11-slim
WORKDIR /app
COPY requirements.txt .
RUN pip install -r requirements.txt
COPY scripts/producer.py scripts/seed.py dashboard_utils.py ./
CMD ["uvicorn", "producer:app", "--host", "0.0.0.0", "--port", "5000"]
```

نقطة مهمة لازم تفهمها: نفس الـ Dockerfile (نفس الـ image) مستخدم لـ 4 خدمات مختلفة (`producer-api`, `db-seed`, `agent`, `consumer`)! كل واحدة منهم بتفرق بس بالـ `command:` يلي بيكتبها بالـ compose file (override للـ CMD الافتراضي). هاد نمط شائع: بناء صورة وحدة فيها كل الاعتماديات، واستخدامها بأكتر من دور حسب الحاجة — بيوفر وقت بناء ومساحة.

ليش `volumes: - ./:/app` على consumer/agent/db-seed بس؟

لأنه الـ `COPY` بالـ Dockerfile بينسخ بس الملفات المحددة (`producer.py`, `seed.py`, `dashboard_utils.py`) لجذر `app/` مباشرة، **مش مجلد `/scripts` كامل**. فلو الـ command كتب `python scripts/consumer.py`، هاد المسار مش موجود أصلًا جوا الصورة المبنية! الحل: نعمل mount لكل مجلد المشروع فوق `app/` وقت التشغيل (`app/:/.`)، وهيك أي ملف موجود عندك على القرص (بما فيه `/scripts`) بيصير متاح جوا الكونتينر مباشرة، بغض النظر عن شو انسخ وقت الـ build. هاد كمان بيسهّل التطوير: تعدّل كود بره، وبتنعكس فورًا بالكونتينر بدون إعادة بناء.

`depends_on`

مهم تنتبه: `depends_on` بـ Docker Compose بس بيتأكد إنه الكونتينر **بلش (started)**، مش إنه **جاهز فعليًا (ready)** يستقبل اتصالات. لهيك المشروع معتمد بمكان تاني على حلول يدوية زي `time.sleep(90)` بـ Flink، و`sleep 60` بـ kafka-init، و retry loop بـ `seed.py` — كلها "حلول بسيطة" لمشكلة عدم وجود health check حقيقي. (لو حابب تذكر تحسين بالمناقشة: `condition: service_healthy` + `healthcheck:` أفضل بكتير من `sleep` الثابت).

---

11. عن "التيست" (Testing)

مهم: فحصت المشروع كامل — **ما في test suite رسمي** (ما في pytest، ما في unit tests، ولا حتى مجلد `/tests`).
المشروع معتمد بالكامل على:

1. **اختبار يدوي End-to-End:** تشغّل الستاك كامل بـ `docker compose up`، تبعت طلب حقيقي من صفحة الطلبات أو بـ curl، وتتابع إنه وصل صح لكل مرحلة (Kafka → Flink → Postgres → Dashboard).
2. **فحص كل مكون لحاله عبر الـ logs والواجهات:**
   - Kafka UI (8080) للتأكد الرسالة وصلت الـ topic.
   - Flink Dashboard (8081) للتأكد الـ job شغال وعم يعالج.
   - `psql` استعلام مباشر بـ للتأكد البيانات انخزنت.
   - الداشبورد نفسه كفحص نهائي بصري.

لو المناقشة بتسأل "كيف بتضمن جودة المشروع بدون tests؟" — جواب صادق وواقعي: هاد المشروع مستوى PoC / تعليمي، والاختبار اليدوي كافي لهالحجم، بس بمشروع إنتاجي حقيقي كان لازم يكون فيه على الأقل:

- Unit tests لـ `dashboard_utils.py` (خصوصًا دوال الـ regex parsing والـ aggregation، لأنها منطق نقي بدون side effects، سهلة الاختبار).
- Integration test لـ `/order` endpoint (باستخدام `TestClient` من FastAPI + mock لـ KafkaProducer).
- اختبار للـ Flink SQL query بأدوات زي `Flink Table API Test Harness`.

---

12. خلاصة سريعة لو سألوك "لخّصلي المشروع بدقيقة"

> مشروع Event-Driven Pipeline: صفحة ويب بتبعت طلب لـ FastAPI، يلي بينشره كـ event على Kafka. Apache Flink بيقرا الـ events بشكل مستمر (streaming SQL) ويحوّلها، وينشرها على topic تاني. Consumer Python بسيط بيقرا التوبيك المحوَّل ويخزّنه بـ PostgreSQL. بالتوازي، AI agent بيقرا نفس الـ events الأصلية ويحللها عبر نموذج LLM محلي (Ollama) ويخزّن التحليل بجدول منفصل. كل شي متغلّف بـ Docker Compose، ولوحة تحكم (dashboard) بتسحب البيانات المجمّعة من قاعدة البيانات عبر FastAPI وتعرضها بشكل مرئي.